

UNIVERSIDAD DE COSTA RICA

ESCUELA DE INGENIERÍA ELÉCTRICA

IE0117: PROGRAMACIÓN BAJO PLATAFORMAS ABIERTAS

Reporte

Laboratorio #4

Prof. Carolina Trejos Quirós

Estudiante: Oriana Mora López, Carne B75079

12 de junio de 2026

Índice

1. Introducción	2
2. Implementación y Resultados	2
3. Discusión de Resultados y Conclusiones	6

1. Introducción

Este proyecto tiene como objetivo aplicar conceptos vistos en clase del lenguaje C relacionados con la gestión dinámica de memoria y la manipulación de estructuras de datos mediante punteros.

En el primer ejercicio se trabaja con matrices dinámicas de valores binarios, implementando funciones para la reserva y liberación de memoria; este problema permite analizar cómo se almacenan los datos en memoria y cómo una matriz puede recorrerse como una estructura lineal utilizando únicamente punteros.

En el segundo ejercicio se procesan imágenes en formato PGM mediante el uso de memoria dinámica y punteros. Se realiza la lectura de la imagen, su conversión a blanco y negro, la generación de un negativo y el cálculo de estadísticas básicas.

Para ambos ejercicios se verifica el uso correcto de la memoria dinámica mediante la herramienta Valgrind, esto nos ayuda a verificar la ausencia de fugas de memoria.

Enlace del repositorio de git: https://github.com/oriimora/Lab-4_Progra-I-2026

Usuario de git: oriimora

2. Implementación y Resultados

Ejercicio 1:

En este ejercicio se trabajó con una matriz cuadrada de valores binarios (0 y 1) creada dinámicamente en memoria. Debíamos encontrar la secuencia consecutiva de unos (1) más larga dentro de la matriz.

Uso funciones `findLargestLine`, `allocateMatrix` y `fillMatrix`.

Paso 1: La función `allocateMatrix` se utilizó para crear la matriz dinámicamente. Primero se reservó memoria para las filas y luego para cada una de las columnas.

Paso 2: La función `fillMatrix` llenó la matriz con valores aleatorios de 0 y 1 utilizando `rand() % 2`. Para acceder a cada elemento se utilizó aritmética de punteros.

Paso 3: La función `printMatrix` recorrió toda la matriz y mostró los valores en pantalla para poder visualizar los datos generados.

Paso 4: La función `findLargestLine` fue la encargada de encontrar la secuencia más larga de 1s consecutivos. Para hacerlo, la matriz se recorrió como si fuera un solo arreglo lineal. Se utilizó una variable llamada `.actual` para contar los 1s consecutivos y otra variable llamada `"mayor"` para guardar la secuencia más larga encontrada.

Paso 5: La función `freeMatrix` se encargó de liberar toda la memoria que se había utilizado para crear la matriz. Esto es importante para que el programa no deje memoria ocupada innecesariamente después de terminar su ejecución.

Paso 6: A la hora de compilar tenemos el siguiente resultado

```

orimora@Orim:~/Escritorio$ gcc ejercicio1.c -o ejercicio1
orimora@Orim:~/Escritorio$ ./ejercicio1
El tamaño de la secuencia de 1s mas grande en: 32766
orimora@Orim:~/Escritorio$ gcc ejercicio1.c -o ejercicio1
orimora@Orim:~/Escritorio$ ./ejercicio1
Ingrese el tamaño de la matriz: 5

Matriz (5x5):
1 0 1 0 0
1 1 0 1 0
0 0 1 1 0
1 1 1 1 0
1 0 0 0 1

La secuencia mas larga de 1s es: 4
orimora@Orim:~/Escritorio$

```

Figura 1: Resultado 1 [?].

Pseudocódigo

Inicio

Para obtener un puntero al primer elemento de la matriz

Primero definir las variables de entrada:

actual = 0

mayor = 0

Para cada elemento de la matriz hacer:

Si el elemento es igual a 1 entonces aumentar actual en 1

Si actual es mayor que mayor entonces mayor = actual

Si no actual = 0

Fin para guardar mayor en resultado

Fin

Valgrind

Después de terminar el programa, se utilizó Valgrind para comprobar que toda la memoria reservada dinámicamente fuera liberada correctamente.

La verificación se realizó con el siguiente comando: "valgrind --leak-check=full ./ejercicio1"

Al ejecutar la prueba, Valgrind indicó que no existían fugas de memoria (memory leaks). Esto significa que toda la memoria reservada mediante malloc fue liberada correctamente mediante la función 'freeMatrix'.

```

orimora@Orim:~/Escritorio$ valgrind --leak-check=full ./ejercicio1
==20956== Memcheck, a memory error detector
==20956== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==20956== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==20956== Command: ./ejercicio1
==20956==
Ingrese el tamaño de la matriz: 5

Matriz (5x5):
1 0 1 0 0
0 1 1 1 1
1 1 1 1 0
0 0 1 0 1
0 0 1 1 0

La secuencia mas larga de 1s es: 8
==20956==

```

Figura 2: Resultado 2 [?].

```

orimora@orivm:~/Escritorio$ valgrind --leak-check=full ./ejercicio1
0 1 1 1 1
1 1 1 1 0
0 0 1 0 1
0 0 1 1 0

La secuencia mas larga de 1s es: 8
==20956==
==20956== HEAP SUMMARY:
==20956==    in use at exit: 0 bytes in 0 blocks
==20956==   total heap usage: 8 allocs, 8 frees, 2,188 bytes allocated
==20956==
==20956== All heap blocks were freed -- no leaks are possible
==20956==
==20956== For lists of detected and suppressed errors, rerun with: -s
==20956== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

Figura 3: Resultado 3 [?].

Se realizan pruebas con diferentes tamaños de matrices:

```

orimora@orivm:~/Escritorio$ valgrind --leak-check=full ./ejercicio1
Ingrese el tamaño de la matriz: 9

Matriz (9x9):
0 1 0 0 0 1 1 1 1
0 1 0 0 0 0 0 0 1
1 0 0 1 1 1 0 0 1
0 0 0 0 0 1 0 0 1
0 0 0 1 0 1 1 1 1
1 1 0 0 1 0 0 0 0
1 0 0 0 0 1 0 0 1
0 1 0 1 1 0 0 0 0
1 1 1 1 0 1 1 0 0

La secuencia mas larga de 1s es: 6
==21477==
==21477== HEAP SUMMARY:
==21477==    in use at exit: 0 bytes in 0 blocks
==21477==
orimora@orivm:~/Escritorio$ valgrind --leak-check=full ./ejercicio1
La secuencia mas larga de 1s es: 6
==21477==
==21477== HEAP SUMMARY:
==21477==    in use at exit: 0 bytes in 0 blocks
==21477==   total heap usage: 12 allocs, 12 frees, 2,444 bytes allocated
==21477==
==21477== All heap blocks were freed -- no leaks are possible
==21477==
==21477== For lists of detected and suppressed errors, rerun with: -s
==21477== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

Figura 4: Resultado 4 [?].

```

primora@OrivM:~/Escritorio$ valgrind --leak-check=full ./ejercicio1
==21839== Memcheck, a memory error detector
==21839== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==21839== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==21839== Command: ./ejercicio1
==21839==
Ingrese el tamaño de la matriz: 14

Matriz (14x14):
0 1 0 1 0 0 0 1 0 1 1 1 0 0
0 0 0 0 0 1 0 0 0 1 0 1 1 1
1 0 0 1 1 0 1 1 0 1 0 1 0 1
0 0 1 0 0 0 0 0 1 0 0 0 0 0
1 1 0 0 1 0 0 0 0 1 1 0 0 1
1 0 0 0 0 0 0 0 0 1 0 1 1 0
1 1 0 1 1 0 1 0 0 1 1 0 0 0
1 0 0 0 0 0 0 0 0 1 0 0 0 0
0 1 0 1 1 1 0 0 1 0 0 0 1 1
0 0 0 1 0 0 0 1 0 0 1 1 1 0
1 1 0 1 1 1 1 0 0 1 0 1 1 0
1 1 0 0 1 0 1 1 0 1 0 0 0 0
1 1 0 1 1 0 0 0 1 1 0 1 1 0
1 0 0 1 0 0 0 0 0 0 0 0 1 0

La secuencia mas larga de 1s es: 4
==21839==
==21839== HEAP SUMMARY:
==21839==    in use at exit: 0 bytes in 0 blocks
==21839== total heap usage: 17 allocs, 17 frees, 2,944 bytes allocated
==21839==
==21839== All heap blocks were freed -- no leaks are possible
==21839==
==21839== For lists of detected and suppressed errors, rerun with: -s
==21839== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
primora@OrivM:~/Escritorio$

```

Figura 5: Resultado 5 [?].

Ejercicio 2:

En este ejercicio se trabajó con imágenes en formato PGM (P2). El programa lee una imagen desde un archivo, aplica un filtro de umbral, genera su negativo, calcula algunas estadísticas y guarda los resultados en nuevos archivos.

Lectura de la imagen

La primera parte del programa es leer la imagen desde el archivo dado que se llama "input.pgm"; de este archivo se obtiene la información básica de la imagen como su tamaño (ancho y alto). Después de leer los datos de la imagen, los píxeles se guardan en un arreglo dinámico usando malloc para poder trabajar con ellos en el programa.

Umbralización de la imagen

Después se aplica un filtro de umbral a la imagen cuando el usuario ingresa un valor T y cada píxel se compara con ese número. Si el píxel es mayor o igual a T, se cambia a 255 (blanco). Si es menor, se cambia a 0 (negro).

Generación de imagen

Después de aplicar el filtro de umbral, se genera una segunda imagen a partir de la primera, cambiando cada píxel para obtener su negativo usando la fórmula $255 - \text{pixel}$. Luego, tanto la imagen en blanco y negro como la imagen negativa se guardan en archivos nuevos en formato PGM, para poder abrirlos y ver los resultados. El programa guarda dos archivos nuevos:

- 1) $\text{output}_{\text{threshold.pgm}}(\text{imagen en blanco y negro})$
- 2) $\text{output}_{\text{negative.pgm}}(\text{imagen negativa})$

```

orimora@OrivM:~$ cd Escritorio
orimora@OrivM:~/Escritorio$ gcc -Wall ejercicio2.c -o ejercicio2
orimora@OrivM:~/Escritorio$ ./ejercicio2
Error leyendo la imagen
orimora@OrivM:~/Escritorio$ gcc -Wall ejercicio2.c -o ejercicio2
orimora@OrivM:~/Escritorio$ ./ejercicio2
Error leyendo la imagen
orimora@OrivM:~/Escritorio$ ./ejercicio2
Ingreso umbral T: 100
Blancos: 51
Negros: 13
Promedio original: 167.89
orimora@OrivM:~/Escritorio$ ./ejercicio2
Ingreso umbral T: 150
Blancos: 43
Negros: 21
Promedio original: 167.89

```

Figura 6: Resultado 6 [?].

Valgrind

Al finalizar el programa, se utilizó Valgrind para comprobar que no hubiera errores de memoria. La verificación se realizó con el siguiente comando: "valgrind --leak-check=full ./ejercicio1"

Esto se hizo para asegurarse de que toda la memoria que se reservó con malloc también fuera liberada correctamente.

```

orimora@OrivM:~/Escritorio$ valgrind --leak-check=full ./ejercicio2
==8441== Memcheck, a memory error detector
==8441== Copyright (c) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==8441== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==8441== Command: ./ejercicio2
==8441==
Ingreso umbral T: 225
Blancos: 7
Negros: 57
Promedio original: 167.89
==8441==
==8441== HEAP SUMMARY:
==8441==   in use at exit: 0 bytes in 0 blocks
==8441== total heap usage: 11 allocs, 11 frees, 15,944 bytes allocated
==8441==
==8441== All heap blocks were freed -- no leaks are possible
==8441==
==8441== For lists of detected and suppressed errors, rerun with: -s
==8441== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
orimora@OrivM:~/Escritorio$

```

Figura 7: Resultado 7 [?].

3. Discusión de Resultados y Conclusiones

Discusión de resultados

En este trabajo se lograron implementar dos programas diferentes utilizando el lenguaje C y el manejo de memoria dinámica.

En el primer ejercicio creó una matriz de números 0 y 1, se logra recorrer usando punteros y se pudo encontrar la secuencia más larga de 1s consecutivos, se realizan tres pruebas con valores diferentes. Los resultados fueron correctos y se observó cómo una matriz puede tratarse como un arreglo lineal.

En el segundo ejercicio se trabajó con imágenes en formato PGM. Se logró leer correctamente la imagen, aplicar un filtro de umbral, generar una imagen en blanco y negro y también su versión negativa, archivos se encuentran adjuntos en el repositorio de git. Además, los resultados cambiaban correctamente dependiendo del valor de umbral ingresado.

En ambos ejercicios se verificó el correcto uso de la memoria con Valgrind, confirmando que no hubo pérdidas de memoria en ambos casos.

Conclusiones

Con este trabajo se logró aplicar de forma práctica conceptos importantes del lenguaje C vistos en clase, como el uso de punteros y la memoria dinámica. En ambos ejercicios se pudo entender la importancia de la correcta gestión de la memoria, ya que liberar lo que se reserva es fundamental para evitar errores en el programa.

En el primer ejercicio se entendió cómo trabajar con matrices dinámicas y cómo recorrerlas sin usar corchetes a trves de punteros, además de aprender a tratar una matriz como un arreglo lineal para resolver problemas de mejor manera o más óptima.

En el segundo ejercicio se reforzó el manejo de archivos y el procesamiento de una manera muy básica de imágenes aplicando operaciones como el filtro de umbral y la generación de imágenes negativas.

Recomendaciones

Por último, se recomienda usar siempre herramientas como Valgrind para asegurarse de que el programa no tenga errores de memoria. Y esta herramineta es práctico usarlo antes de hacer commit en git para asegurarnos que todo se encuentra funcioanndo de la mejor manera.